RAG Q&A; Practice Guide

A sample knowledge document for learning Retrieval-Augmented Generation (RAG), document ingestion, chunking, embeddings, vector search, retrieval, prompting, and evaluation.

1. What is Artificial Intelligence?

Artificial Intelligence (AI) is the field of building computer systems that perform tasks that normally require human-like capabilities such as perception, reasoning, language understanding, prediction, and decision making. Modern AI systems commonly use machine learning models trained on data.

2. Machine Learning

Machine learning is a branch of AI in which models learn patterns from examples rather than being programmed with a separate rule for every situation. Supervised learning uses labeled examples, while unsupervised learning discovers patterns in unlabeled data. Evaluation should use data that was not used to train the model so that performance can be measured on unseen examples.

3. Deep Learning and Neural Networks

Deep learning uses neural networks with multiple layers. These networks can learn increasingly abstract representations from raw inputs. Transformers are a neural-network architecture that became particularly important for natural language processing because they can model relationships between tokens using attention mechanisms.

4. Natural Language Processing

Natural Language Processing (NLP) focuses on processing and generating human language. Common NLP tasks include classification, summarization, translation, question answering, information extraction, and semantic search. Text is usually converted into numerical representations before machine-learning models can process it.

5. Large Language Models

Large Language Models (LLMs) are neural language models trained on large collections of text and other data. They can generate answers, summarize documents, transform text, and perform many language tasks. An LLM's generated answer is not automatically a verified fact. When current or private information is needed, an external knowledge source can be supplied to the model.

6. Embeddings

An embedding is a numerical vector representing the semantic characteristics of an item such as a sentence, paragraph, document, image, or query. Texts with related meanings tend to have vectors that are close according to an appropriate similarity metric. Embeddings are widely used for semantic search.

7. Vector Databases

A vector database stores vectors and metadata and provides similarity-search operations. Examples of systems commonly used for vector search include FAISS, Chroma, Pinecone, Weaviate, Milvus, and pgvector. A practical RAG system may store each text chunk's embedding together with metadata such as source name, page number, section, or document identifier.

8. What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with text generation. Instead of asking an LLM to answer only from information encoded in its parameters, a RAG application first retrieves relevant passages from an external knowledge base and then provides those passages to the model as context.

9. Typical RAG Pipeline

Stage	Purpose
1. Load	Read PDFs, web pages, text files, or other source documents.
2. Extract	Convert source files into usable text and preserve useful metadata.
3. Chunk	Split long documents into smaller passages suitable for retrieval.
4. Embed	Convert each chunk into a numerical embedding vector.
5. Index	Store embeddings and metadata in a vector or hybrid search system.
6. Retrieve	Search for passages relevant to the user's question.
7. Prompt	Place retrieved passages and the question into an LLM prompt.
8. Generate	Produce an answer grounded in the retrieved context.
9. Evaluate	Measure retrieval quality, answer quality, and citation/grounding behavior.

10. Chunking

Chunking divides a document into smaller units before embedding. Fixed-size chunking is simple but can split ideas awkwardly. Recursive or structure-aware chunking attempts to preserve paragraphs, headings, or other meaningful boundaries. Chunk overlap repeats a small amount of text between neighboring chunks, which can help preserve context across boundaries. Very large chunks may reduce retrieval precision, while very small chunks can lose useful context.

11. Retrieval

Given a user question, the application creates a query representation and searches the knowledge base. Vector similarity can retrieve semantically related text even when the wording differs. Keyword or lexical search can be useful for exact names, identifiers, and rare terms. Hybrid retrieval combines semantic and lexical signals. Reranking can then reorder an initial set of retrieved passages using a more expensive model.

12. Prompt Grounding

A grounded RAG prompt should clearly separate the user's question from retrieved context. A common instruction is to answer using the supplied context and to say that the information is not available when the context does not support an answer. Retrieved passages can also be labeled with source metadata so that the application can display citations.

13. RAG Failure Modes

RAG can fail even when the language model itself is capable. Common problems include poor text extraction, bad chunk boundaries, irrelevant retrieval, missing information, duplicate chunks, insufficient retrieval depth, weak prompts, context-window limits, and incorrect source attribution. Therefore, improving a RAG system usually requires examining both retrieval and generation rather than changing only the LLM.

14. RAG Evaluation

RAG evaluation can be divided into retrieval and generation. Retrieval metrics examine whether relevant documents or chunks were returned. Generation evaluation examines whether the final answer is correct, supported by the retrieved context, relevant to the question, and sufficiently complete. A useful test set contains questions with known answers, questions requiring multiple passages, and questions for which the correct response is that the document does not contain enough information.

15. Practice Questions

1. What does RAG stand for?
2. Why might an application use RAG instead of relying only on an LLM?
3. What is an embedding?
4. What is the purpose of a vector database?
5. Name three systems that can be used for vector search.
6. What happens during the chunking stage?
7. Why can chunk overlap be useful?
8. What is hybrid retrieval?
9. What is reranking?
10. What metadata might be stored with a text chunk?
11. List the major stages of a typical RAG pipeline.
12. Why can very small chunks be problematic?
13. Why can very large chunks be problematic?
14. What is the difference between retrieval quality and generation quality?
15. Give two examples of RAG failure modes.
16. Why should an RAG prompt distinguish user questions from retrieved context?
17. What should a grounded application do when the retrieved context does not support an answer?
18. Why can keyword search still be useful when vector search is available?
19. What is the role of embeddings in semantic search?
20. What does a reranker do?
21. What kinds of documents can a RAG application ingest?
22. Why is source metadata useful for citations?
23. Why should a RAG evaluation set include questions with no supported answer?
24. What is one reason an LLM's generated answer should not automatically be treated as a verified fact?
25. Which two broad parts of a RAG system should be evaluated?

16. Answer Key

1. Retrieval-Augmented Generation.
2. RAG lets the application provide external or private knowledge to the model at query time.
3. A numerical vector representing semantic characteristics of an item such as text.
4. To store vectors and metadata and perform similarity search.
5. FAISS, Chroma, Pinecone, Weaviate, Milvus, or pgvector.
6. It splits source text into smaller passages for embedding and retrieval.

7. It can preserve context that crosses chunk boundaries.

8. A retrieval approach combining semantic/vector and lexical/keyword signals.

9. It reorders initially retrieved passages to put the most useful results first.

10. Source name, page number, section, or document identifier.

11. Load, extract, chunk, embed, index, retrieve, prompt, generate, and evaluate.

12. They can lose surrounding context needed to understand an idea.

13. They may reduce retrieval precision and make it harder to fit useful results into the prompt.

14. Retrieval quality asks whether relevant evidence was found; generation quality asks whether the answer is correct and grounded.

15. Examples include poor extraction, bad chunking, irrelevant retrieval, missing information, weak prompts, and incorrect attribution.

16. Clear separation helps the model use retrieved material as evidence rather than confusing it with the question.

17. It should state that the available context does not support the answer rather than inventing unsupported information.

18. Keyword search is useful for exact names, identifiers, and rare terms.

19. They convert text into vectors that can be compared for semantic similarity.

20. It reorders retrieved candidates using a more detailed relevance model.

21. PDFs, web pages, text files, and other source documents.

22. It allows the application to identify where an answer came from.

23. It tests whether the system can avoid inventing an answer when evidence is absent.

24. LLMs generate text probabilistically and their output is not automatically verified against an external source.

25. Retrieval and generation.

17. Suggested Mini Project

Build a small PDF Q&A; application. Load this PDF, extract its text, split it into chunks, create embeddings, store the vectors in FAISS or Chroma, retrieve the top relevant chunks for a question, and send those chunks to an LLM. Display the answer together with the source chunk or page information. Then test the system using the 25 questions above and add a few questions whose answers are intentionally absent from the document.